SAFETY-CRITICAL SOFTWARE - BONUS ASSIGNMENT

MedSafe Race Lab

A Java demonstration of a medical race condition and three correct synchronization solutions

Prepared submission

This report answers the race-condition assignment, documents an executable Java example, compares three protection mechanisms, and connects the result to safety-critical engineering.

Submitter 1	Full name: _____	ID: _____
Submitter 2	Full name: _____	ID: _____
Submitter 3	Full name: _____	ID: _____
Course	BIO-MED SW / Software as a Medical Device	Date: _____

Safety disclaimer

The application is a fictional educational simulator. It does not calculate, recommend, or deliver medical treatment and must never be used for clinical decisions.

Project package: Java 17, JavaFX, Maven, JUnit 5

Demo login: student / MedSafe2026!

1. Executive summary

A race condition is dangerous when a safety decision depends on a shared state that another task can change between the decision and its use. MedSafe Race Lab models this risk with two concurrent Java tasks: an operator changes a treatment configuration while a treatment controller attempts to start.

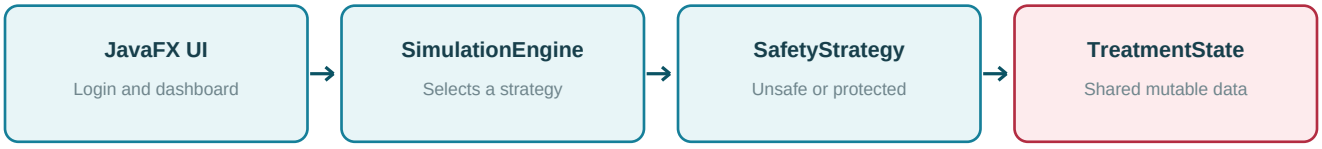
The unsafe implementation allows the controller to observe an intermediate X-ray state: the old verification flag is still true, but the beam-spreading target has not yet been placed. The same scenario is then protected using synchronized, ReentrantLock, and Semaphore(1).

> **Verified result**
>
> The controlled unsafe case produces one safety violation per run. Across 25 repeated runs, each of the three protected implementations produces zero violations.

Assignment coverage

Requirement	Where it is answered
Define race condition	Section 2
Demonstrate the bug in Java	Sections 3-4 and executable project
Explain correct solutions	Sections 5-7
Find fatality statistics	Section 9
Identify common circumstances	Section 10

Application architecture



The scenario stays constant. Only the synchronization boundary changes.

2. What exactly is a race condition?

> **Definition**
>
> A race condition occurs when a program result depends on the unpredictable timing or interleaving of concurrent tasks that access shared mutable state, with at least one write, without adequate synchronization.

Three necessary ingredients

Ingredient	Meaning in this project
Shared mutable data	OperatorThread and TreatmentThread access one TreatmentState.
At least one write	The operator changes mode, energy, target position, and verification.
No atomic boundary	Treatment can enter after one field changes but before the full update finishes.

Atomicity and the check-then-act problem

An operation is atomic when other tasks cannot observe it halfway through. In the unsafe code, checking verified and using the treatment configuration do not form one protected transaction. This is a time-of-check to time-of-use race.

Safety invariant

- X_RAY mode requires the beam-spreading target to be in place and an allowed energy.
- ELECTRON mode requires the X-ray target to be outside the beam path.
- Treatment may start only after the complete configuration has been verified.

> **Important distinction**
>
> Thread existence alone is not a race condition. The bug appears because multiple tasks access the same mutable state without a complete synchronization boundary.

3. Therac-25 and safety-critical context

The course material presents Therac-25 as a classic safety-critical software disaster. The original investigation by Leveson and Turner documents six known massive-overdose accidents between 1985 and 1987, with deaths and serious injuries. The course slide reports three patient deaths.

The investigation describes concurrent access to shared memory, no real synchronization beyond shared variables, and test/set behavior that was not indivisible. Race conditions played an important part in the accidents.

Why this must not be reduced to one bad line of code

- Software replaced or carried responsibilities previously enforced by hardware interlocks.
- The safety analysis initially did not treat software as a source of hazardous failure.
- Some failures depended on fast operator editing and were difficult to reproduce.
- Error messages were cryptic and operators were accustomed to non-critical pauses.
- Incident reports and corrective action were not handled quickly or comprehensively enough.

> **Scope of the implemented scenario**
>
> MedSafe Race Lab is a conceptual Java teaching model inspired by the concurrency lessons. It is not a literal reconstruction of Therac-25 software or radiation physics.

SaMD relevance

The supplied FDA/IMDRF SaMD guidance emphasizes that incorrect software output can affect clinical management. It also separates valid clinical association, analytical/technical validation, and clinical validation, and treats evaluation as an ongoing lifecycle activity. This project concentrates on the technical validation of one concurrency safety property.

4. Unsafe Java demonstration

The treatment begins in a safe ELECTRON configuration: 10 MeV, target outside the beam path, and verified=true. OperatorThread begins converting it to X_RAY by writing several fields. It changes the mode before invalidating the previous verification flag.

```
// No common lock protects the complete update.
void changeToXRayUnsafe() {
    state.mode = X_RAY;          // intermediate unsafe state begins
    state.verified = false;
    state.energyMeV = 25;
    state.targetInPlace = true;
    state.verified = true;
}

void startTreatmentUnsafe() {
    if (state.verified) {        // stale TRUE may be observed
        beamOn(state);           // state can be internally inconsistent
    }
}
```

Controlled interleaving

Step	Task	Action	Observed state
1	OperatorThread	mode = X_RAY	X_RAY, 10 MeV, target=false, verified=true
2	TreatmentThread	Reads shared state	The stale verified flag still permits start
3	TreatmentThread	Starts simulated treatment	Safety invariant is violated
4	OperatorThread	Finishes the remaining writes	Final state becomes safe, but too late

Why CountDownLatch appears in the demonstration

The latch is used only by the demonstration harness to reproduce the dangerous schedule every time. It does not protect TreatmentState. The unsafe read and write remain outside a mutual-exclusion boundary. This prevents the classroom demo from succeeding only by luck.

> **Unsafe observed snapshot**
>
> mode=X_RAY, energy=10 MeV, targetInPlace=false, verified=true, revision=2. The controller starts because verification is stale even though the configuration is not safe.

5. Solution 1 - synchronized

Java's intrinsic monitor allows only one task at a time to execute a block synchronized on the same monitor object. The monitor is released automatically when execution leaves the block, including exceptional exit.

```
private final Object monitor = new Object();

void changeToXRay() {
    synchronized (monitor) {
        updateCompleteTreatmentConfiguration();
    }
}

void startTreatment() {
    synchronized (monitor) {
        if (state.verified && state.isSafe()) {
            beamOn(state);
        }
    }
}
```

Why it works

TreatmentThread reaches the race boundary while OperatorThread owns the monitor. It must wait. After the operator completes all fields and exits the block, treatment reads one consistent snapshot.

Advantages and limitations

Advantages	Limitations
Concise; automatic release; strong visibility guarantee	Less control over timed or interruptible acquisition
Good default for a simple critical section	Every access must use the same monitor

6. Solution 2 - ReentrantLock

ReentrantLock provides explicit acquisition and release. A fair lock is used in this project so waiting tasks are generally granted access in arrival order. The essential correctness rule is that unlock() must execute in finally.

```
private final ReentrantLock lock = new ReentrantLock(true);

void startTreatment() {
    lock.lock();
    try {
        if (state.verified && state.isSafe()) {
            beamOn(state);
        }
    } finally {
        lock.unlock();           // guaranteed cleanup
    }
}
```

Why finally is mandatory

If an exception occurs after lock(), a missing finally block could leave the lock permanently held. Other tasks would block forever, creating a liveness failure or deadlock-like shutdown.

When ReentrantLock is useful

- Timed acquisition with tryLock(timeout).
- Interruptible waiting with lockInterruptibly().
- Multiple Condition objects for advanced coordination.
- Explicit fairness configuration.

7. Solution 3 - Semaphore(1)

A semaphore controls a number of permits. With exactly one permit, only one task may enter the protected section at a time. This provides mutual exclusion for the demonstration.

```
private final Semaphore mutex = new Semaphore(1, true);

void startTreatment() throws InterruptedException {
    boolean acquired = false;
    try {
        mutex.acquire();
        acquired = true;
        if (state.verified && state.isSafe()) {
            beamOn(state);
        }
    } finally {
        if (acquired) {
            mutex.release();
        }
    }
}
```

Critical exam rule

Semaphore(1) allows one task into the critical section. Semaphore(2) allows two tasks and would not provide mutual exclusion here. release() belongs in finally, but only after acquire() succeeded.

Important difference from a lock

A semaphore does not enforce ownership: one task can release a permit acquired by another. It is also not reentrant. For a simple shared-state critical section, synchronized or ReentrantLock is usually the more natural design; Semaphore(1) is included because it is a valid requested solution.

8. Verification and test results

The project includes eight JUnit tests. They verify authentication, password-array cleanup, the deterministic unsafe interleaving, every protected mode, and a repeated comparison matrix.

Execution mode	Runs	Treatments started	Safety violations	Expected result
Unsafe	25	25	25	Failure exposed
synchronized	25	25	0	Safe
ReentrantLock	25	25	0	Safe
Semaphore(1)	25	25	0	Safe

Test acceptance criteria

- Unsafe treatment starts and the observed snapshot violates the invariant.
- Each protected treatment starts only with a safe observed snapshot.
- The final configuration is safe in every mode.
- Only unsafe runs contain a SAFETY VIOLATION event.
- All eight automated tests finish successfully.

> **What synchronization proves - and what it does not**
>
> Passing these tests proves the implemented concurrency invariant for this scenario. A real medical device would also require hazard analysis, independent safety controls, traceability, formal review, clinical evidence, usability engineering, and regulatory compliance.

9. Software-related fatalities: documented cases

A single global death count would create false precision because accident definitions and causal attribution differ. This report therefore presents incident-level statistics with a visible source and does not add unlike cases into one total.

Incident	Period	Reported fatalities	Software/system lesson	Source
Therac-25	1985-1987	3 in course material; paper reports several	Concurrent shared state plus wider safety-engineering failures	Leveson and Turner
Patriot missile	1991	28	Timing error accumulated after prolonged continuous operation	U.S. GAO IMTEC-92-26
Boeing 737 MAX	2018-2019	346	Systemic design, software, certification, training, and organizational failure	U.S. House investigation
Uber developmental ADS	2018	1	Automation behavior, disabled emergency response, and weak oversight	NTSB HAR-19/03

Interpretation caution

Software-related does not mean software-only

The 737 MAX and Uber cases should not be described as one isolated programming bug. Official investigations describe interacting technical, operational, human, management, and oversight factors.

10. Common reasons and circumstances

Common circumstance	How it increases risk
Single point of catastrophic failure	One software decision can activate a hazardous function without an independent barrier.
Incomplete verification and validation	Tests miss timing boundaries, prolonged operation, realistic workload, or unusual inputs.
Poor human-machine interface	Ambiguous warnings and recovery paths delay correct response.
Weak incident learning	Early reports are dismissed, isolated, or not shared across users.
Schedule and production pressure	Safety review, documentation, and correction receive insufficient time or authority.
Fragmented responsibility	Software, hardware, operations, management, and regulators assume another layer will prevent harm.

Defense-in-depth recommendations

- Make treatment configurations immutable snapshots and publish them atomically.
- Fail closed: any inconsistency must block treatment rather than permit continuation.
- Revalidate immediately before simulated activation, under the same synchronization boundary.
- Use independent hardware or system safety interlocks for catastrophic hazards.
- Produce clear, actionable error messages and a complete audit trail.
- Test concurrency systematically, including deterministic schedules and stress testing.
- Investigate every safety incident across installations and feed lessons into CAPA.

11. Application walkthrough

The interface is designed for a short classroom demonstration. The login provides a clear start, the theory screen explains the concept, and the simulator exposes the exact sequence of thread events.

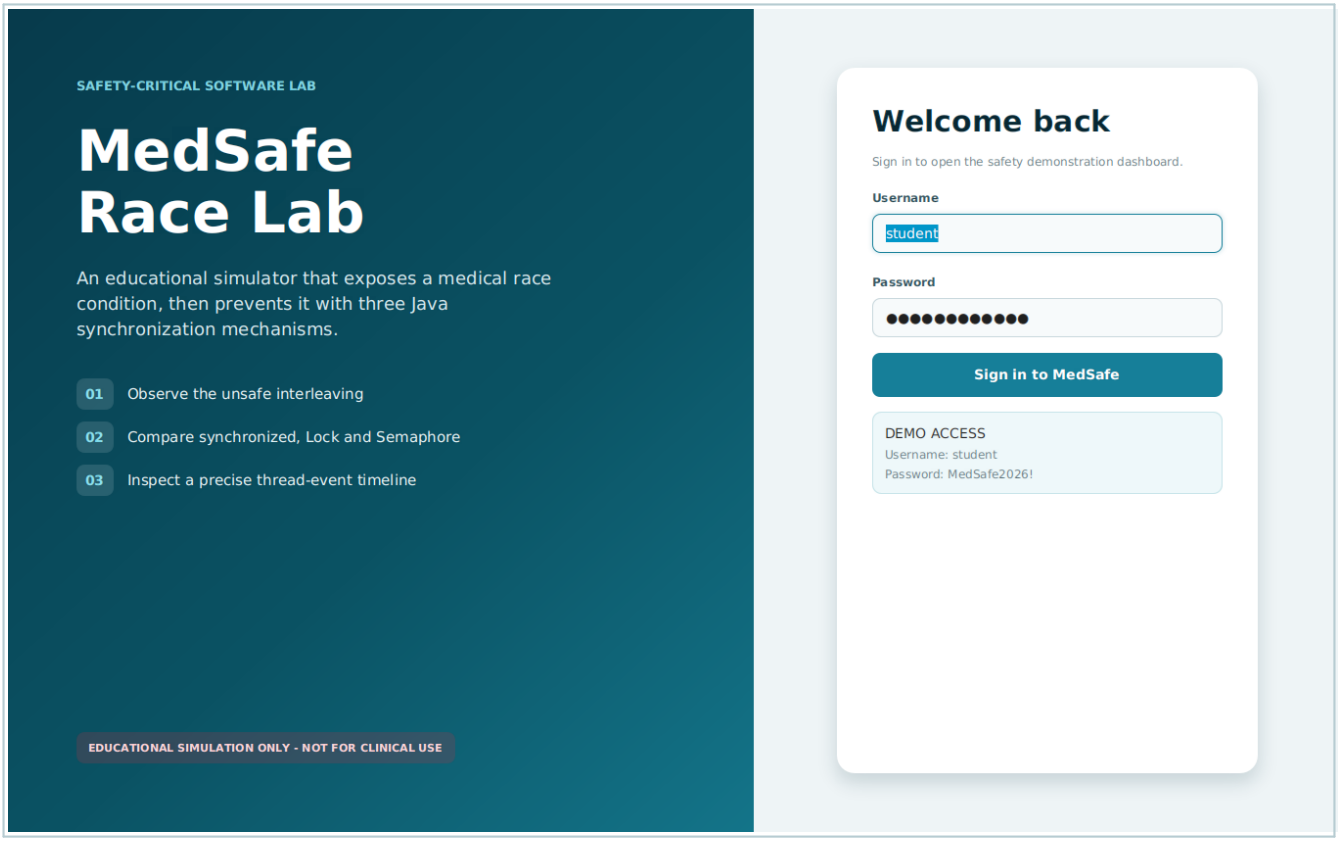


Figure 1. Login screen and educational-use disclaimer.

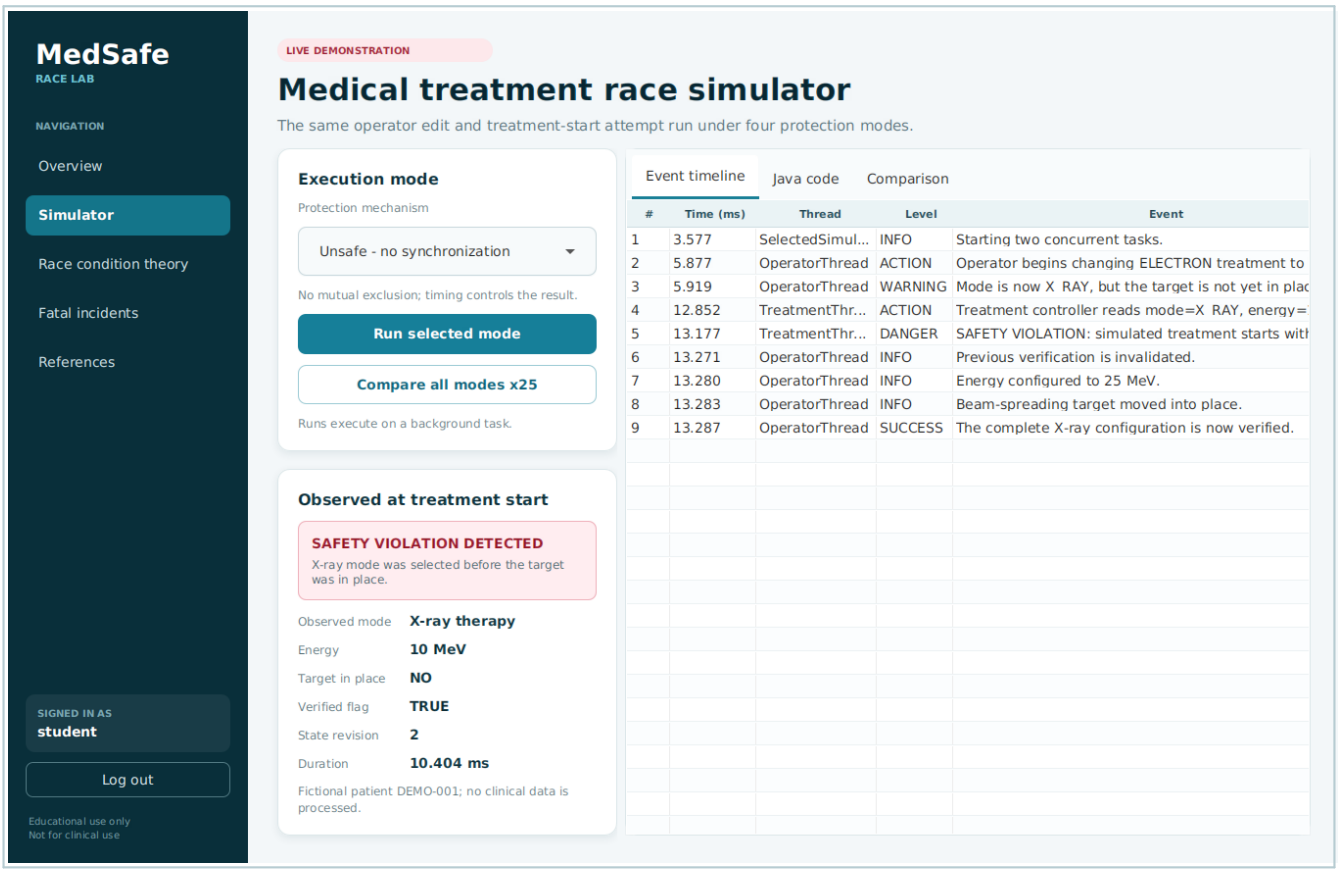


Figure 2. Unsafe mode showing the violated treatment snapshot and event timeline.

12. How to run the project

Prerequisites

- JDK 17 or newer.
- Apache Maven 3.9 or newer.
- VS Code with Extension Pack for Java (recommended).

Commands

```
# From the MedSafe-Race-Lab project folder
mvn clean test
mvn javafx:run
```

Demo credentials

Username	Password
student	MedSafe2026!

Submission package

- Complete CREDITS.TXT with all names and ID numbers.
- Upload the source project to GitHub.
- Submit the GitHub link and this PDF report.
- Do not submit an EXE file, in accordance with the course rules.

> **Database decision**
>
> No SQL database is required. The race is intentionally demonstrated in shared JVM memory. The login is local, passwords are verified with PBKDF2, and no patient information is stored.

13. Conclusion

The unsafe result is not caused by a wrong final configuration. It is caused by allowing another task to observe the configuration while it is only partly updated. This is why testing only the final state is insufficient for concurrent safety-critical software.

All three requested mechanisms prevent the demonstrated race when they protect the complete update and the complete check/use operation with the same synchronization boundary. synchronized is the simplest solution, ReentrantLock provides more control, and Semaphore(1) provides mutual exclusion when permit management is correct.

> **Final engineering lesson**
>
> Thread safety is necessary but not sufficient for medical safety. High-assurance systems also need defense in depth, clear requirements, independent safeguards, verification, validation, traceability, and rapid learning from incidents.

References

[1] BIO-MED SW COURSE 2025, Unit 1: Software as a Medical Device, slides 9-10 and 18-25.

[2] FDA / IMDRF. *Software as a Medical Device (SaMD): Clinical Evaluation*. 2017.

[3] Leveson, N. G. and Turner, C. S. An Investigation of the Therac-25 Accidents. 1992/1993.

[4] U.S. GAO. Patriot Missile Defense: Software Problem Led to System Failure at Dhahran. GAO/IMTEC-92-26, 1992.

[5] U.S. House Committee on Transportation and Infrastructure. Boeing 737 MAX Investigation. 2020.

[6] National Transportation Safety Board. Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian. HAR-19/03, 2019.

[7] Oracle. Java SE 17 API documentation for ReentrantLock, Semaphore, and CountDownLatch.